🎓 QA101 — n8n Quickstart

n8n Academy · Selesai · Updated 2026-11-09

Ringkasan

Catatan kursus **n8n Quickstart** dari n8n Academy: dasar workflow, Data Table, filtering, Code node, penjadwalan, ekspor/impor, pengolahan data, hingga membangun **AI Agent** dengan memory dan tools.

Peta Materi

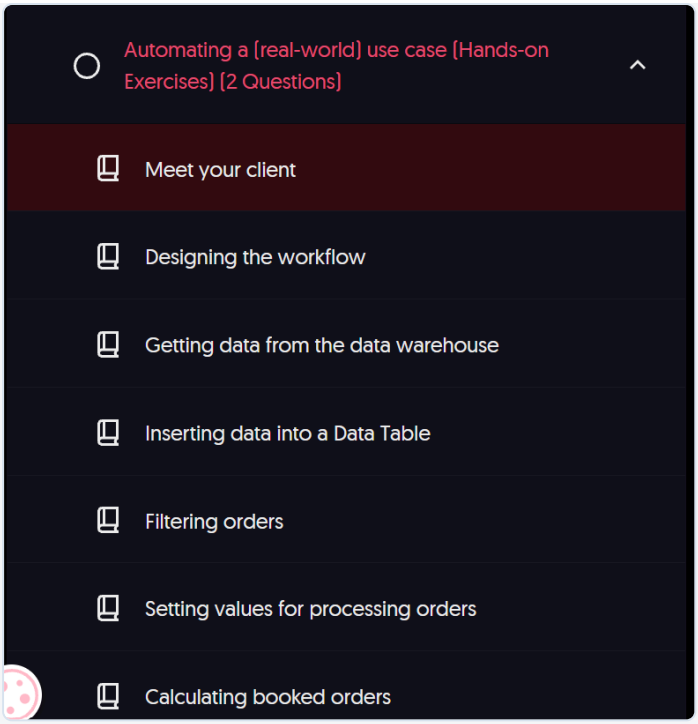
Bagian	Fokus	Unit
PART 1	Getting Started — workflow, Data Table, filter, Code, Discord, schedule, publish, export/import	1–10
PART 2	Working with Data — struktur data, HTML/XML, tanggal, binary, split, merge, loop, debugging, error handling, latihan	1–12
PART 3	Building an AI Agent — memory, tools, system prompt, pengujian	1–11
Bank Soal	Kuis & ujian QA101 (16 soal)	—

🗺️ Isi Catatan

1. **PART 1 — Getting Started**: desain workflow, Data Table, filter, Edit Fields, Code node, Discord, Schedule Trigger, publish, export/import.
2. **PART 2 — Working with Data**: struktur data, HTML/XML, tanggal & waktu, binary data, split, merge, looping, debugging, error handling, dan latihan.
3. **PART 3 — Building an AI Agent**: masalah memory, Simple Memory, tools, system prompt, dan pengujian agent.
4. **Bank Soal & Penutup**: 16 soal kuis beserta jawaban, ringkasan program lanjutan, dan cheat sheet node.

Pembuka

Berawal dari penasaran, mari kita selami.



Jadi kita benar-benar tahu alurnya bagaimana.

PART 1 — Getting Started

1. Meeting Part 1 — Siapa Nathan?

👤 Siapa Nathan?

Seorang karyawan yang menghadapi tugas mingguan yang sangat melelahkan, berulang-ulang, dan membuat jenuh (*repetitive and mind-numbing*).

⚠️ Apa Masalah Utamanya?

Setiap minggu, Nathan harus mengumpulkan data penjualan dari *data warehouse* perusahaan yang merupakan sistem lama (*legacy*) buatan internal (*in-house*) dan **tidak memiliki fitur ekspor CSV**.

📋 Apa Tugas Rutinnya?

1. Menghitung total pesanan yang berstatus `Booked` untuk diumumkan di Discord perusahaan setiap hari Senin.
2. Menyusun daftar pesanan berstatus `Processing` agar para Manajer Penjualan bisa meninjaunya.

🚨 Kendala & Risiko Pekerjaan

Karena dikerjakan secara manual dan menuntut ketelitian tinggi (*attention to detail*), pekerjaan ini sangat rentan kesalahan (*error-prone*). Nathan sering kehilangan fokus, salah mengetik angka, terlambat menyelesaikannya, hingga pernah ditegur oleh manajernya.

---

2. Designing the Workflow — Part 2

🎯 Tujuan Utama Workflow

Membangun otomatisasi **8-node** untuk mengolah data penjualan mingguan dari *legacy data warehouse* internal tanpa fitur ekspor CSV.

🛠️ Rincian 6 Langkah Alur Kerja

#	Langkah	Fungsi
1	**Get Data**	Mengambil data penjualan (*order ID*, *status*, *value*, *employee name*) dari *data warehouse* via API.
2	**Filter Data**	Memisahkan pesanan berdasarkan status (`Processing` vs `Booked`).
3	**Calculate Sum**	Menghitung total nilai nominal dari seluruh pesanan berstatus `Booked` .
4	**Discord Notification**	Mengirim hasil laporan pesanan `Booked` ke saluran Discord perusahaan.
5	**Data Table Storage**	Memasukkan detail pesanan `Processing` ke n8n Data Tables untuk proses *follow-up* Manajer Penjualan.

#	Langkah	Fungsi
6	**Schedule Automation**	Menjadwalkan *workflow* agar berjalan otomatis setiap Senin pagi.

🌐 Konsep Utama Integrasi: HTTP Request Node

> **Mengapa menggunakan HTTP Request Node?**
>
> Karena sistem *data warehouse* lama tidak memiliki **Dedicated Node** (node integrasi khusus seperti Discord atau Google Sheets), maka **HTTP Request Node** digunakan sebagai solusi serbaguna (*versatile*) untuk berkomunikasi langsung dengan titik akses **API Endpoint** sistem tersebut.

⚙️ Konfigurasi Node `GetDataFromWarehouse`

Parameter	Pengaturan / Nilai
Node Type	HTTP Request (terhubung ke *Manual Trigger*)
Method	`GET`
URL	`https://learn.app.n8n.cloud/webhook/courses/n8n-quickstart/company-warehouse`
Send Headers	`true` (*Using Fields Below*)
Header Name	`x-assessment-id`
Header Value	`<ID_ASSESSMENT_KAMU>`
Authentication	`Generic Credential Type`
Generic Auth Type	`Header Auth`
Credential Setup	Name: `X-API-KEY` | Value: *(Kunci API Latihan)*

---

3. Inserting Data into a Data Table — Part 3

💡 Konsep Utama

- **n8n Data Table**: database internal bawaan (*built-in*) mirip *spreadsheet* yang tersimpan langsung di n8n (*live right inside*).
- **Bebas Integrasi**: tanpa perlu aplikasi luar (*no external services required*) atau kredensial API.
- **Syarat**: *workflow* dan *data table* harus berada di **workspace** yang sama.

⚙️ Konfigurasi Node `UpsertOrders`

Parameter	Nilai
Node	Data Table (dihubungkan dari `GetDataFromWarehouse`)
Operation	`Upsert row(s)` — tambah data baru atau perbarui jika sudah ada
Data Table	`orders`

Parameter	Nilai
Must Match	`Any condition`
Conditions	`orderId` → `equals` → `{{ $json.orderID }}` *(Expression Mode)*
Mapping Mode	`Map Automatically`

🚨 Poin Penting

- **Auto Mapping**: nama *field* dari output HTTP Request wajib **sama persis** dengan nama kolom tabel.
- **Fungsi Upsert**: mencegah duplikasi data (*duplicates*) saat pengujian ulang (*re-run*).
- **Built-in Columns**: kolom otomatis dari n8n (`id` , `createdAt` , `updatedAt`).

> **Next Step**
>
> Menambahkan node **Filter** untuk memisahkan pesanan berstatus `Processing` (masuk ke Data Table) dan `Booked` (dihitung total nilainya).

---

4. Filtering Orders — Part 4

`#n8n` `#workflow` `#automation` `#learning-notes`

> 💡 **Konsep Utama**
>
> - **Filtering Data:** memilah data yang masuk menggunakan **logika bersyarat** (*conditional logic* / aturan *if-then-else*) sebelum dimasukkan ke dalam database atau Data Table.
> - **If Node:** membelah alur *workflow* menjadi **2 cabang** (`True` & `False`) berdasarkan hasil evaluasi perbandingan.
> - **Switch Node:** digunakan jika perlu menyaring data ke **lebih dari 2 kondisi/cabang** (*multiple output connectors*).
> - **Efisiensi Data (Best Practice):** hanya simpan *field/kolom* yang benar-benar dibutuhkan. Menyimpan data tak terpakai memboroskan daya komputasi, memperlambat proses, dan memakan ruang penyimpanan.

⚙️ Konfigurasi Node `CheckOrderStatus`

- **Node Type:** `If`
- **Connected From:** `HTTP Request`

Parameter	Setting / Value
Value 1	`{{ $json.orderStatus }}` *(Expression Mode)*
Operation	`String` → `is equal to`
Value 2	`processing`

🚨 Poin Penting & Dampak Performa

- **Alur Output:**
- True Branch → meneruskan data berstatus `processing` ke **Data Table**.
- False Branch → meneruskan data status lain (misal: `booked`) untuk dihitung nilainya.
- **Optimization Note:** meskipun pada data kecil (misal 14 *records*) dampaknya belum terasa, menambah kolom/data ekstra pada ribuan *records* akan sangat memengaruhi kinerja sistem (*add up and affect performance*).

⏩ Langkah Selanjutnya

Mengonfigurasi cabang **False** untuk menghitung total akumulasi nilai dari pesanan berstatus **Booked**.

---

5. Setting Values for Processing Orders — Part 5

💡 Konsep Utama

- **Edit Fields (Set) Node:** node yang berfungsi untuk memilih, mengubah, menyaring (*filter*), atau menambah data sebelum diteruskan ke node berikutnya (seperti *Data Table* atau *database*).
- **Seleksi Data (Data Reduction):** hanya meneruskan data yang benar-benar dibutuhkan (misal hanya `employeeName` dan `orderID`) dari *processing orders* untuk menjaga efisiensi penyimpanan dan performa.
- **Pilihan Manipulasi Data:** 1. **Mengambil Data Lama:** memilih *field* spesifik dari node terdahulu. 2. **Completely New Data:** membuat *field/kolom* baru dengan nilai tetap (*fixed value*) atau kalkulasi baru yang belum ada sebelumnya. 3. **Overwrite Data:** menimpa/menggantikan nilai data lama yang sudah ada.

⚙️ Konfigurasi & Alur Kerja

- **Node Type:** `Edit Fields (Set)`
- **Connected From:** cabang `True` milik node `CheckOrderStatus` (If Node)
- **Target Fields:** `employeeName` & `orderID`

Fitur / Tindakan	Deskripsi Fungsi
Select / Set Fields	Menentukan *field* mana saja yang diizinkan lewat ke node berikutnya.
Data Clean-up	Membuang kolom-kolom lain yang tidak krusial dari output `HTTP Request`.
Preparation	Menyiapkan struktur data yang rapi sebelum proses *insert/upsert* ke Data Table.

🚨 Poin Penting & Best Practice

- **Data Expectations:** node seperti *spreadsheet* atau *database* membutuhkan struktur data yang presisi. Gunakan Edit Fields node untuk merapikannya terlebih dahulu.

- **Performa & Storage:** membuang *field* yang tidak relevan mengurangi ukuran muatan data (*payload*), sehingga proses enkripsi/transfer data menjadi lebih cepat.

⏩ Langkah Selanjutnya (Next Unit)

Menggunakan **Custom JavaScript Code** di dalam node khusus untuk menghitung dan menjumlahkan total akumulasi pesanan berstatus **Booked** (cabang *False*).

---

6. Calculating Booked Orders (Code Node) — Part 6

💡 Konsep Utama & Mode Operasional Code Node

- **Code Node:** node fleksibel yang memungkinkan kita menjalankan kustomisasi menggunakan **JavaScript** atau **Python**.
- **2 Mode Operasional Code Node:** 1. **Run Once for All Items:** memproses seluruh item input sekaligus dalam satu kelompok (*as a group*). Cocok untuk kalkulasi total, *sum*, atau *aggregation*. 2. **Run Once for Each Item:** menjalankan kode satu per satu secara terpisah untuk setiap baris data input.
- **Struktur Data Asli n8n:** data yang dialirkan antar-node berbentuk **Array of Objects** dengan struktur wajib bernested: `[{ json: { ... }, binary: { ... } }]`.

⚙️ Konfigurasi

- **Node Type:** `Code` (pilih: *Code in JavaScript*)
- **Connected From:** cabang `False` milik node `CheckOrderStatus` (If Node)
- **Execution Mode:** `Run Once for All Items`

Contoh implementasi

```
// Mode: Run Once for All Items
// Sesuaikan nama field dengan output HTTP Request kamu
let total = 0;

for (const item of $input.all()) {
  total += Number(item.json.orderValue);
}

return [{ json: { totalBooked: total } }];
```

Hasilnya berupa **satu item** berisi total nilai pesanan `Booked`, yang siap dikirim ke Discord.

---

7. Notify the Team (Discord) — Part 7

`#n8n` `#workflow` `#automation` `#discord` `#webhook` `#expressions` `#learning-notes`

> 💡 **Konsep Utama**
>
> - **Otomatisasi Notifikasi:** mengirimkan hasil olah data (laporan ringkasan pesanan berstatus *booked*) secara otomatis ke platform komunikasi tim (Discord) untuk menghemat waktu dan menghindari pekerjaan manual yang repetitif (*tedious work*).
> - **Integrasi Webhook:** menggunakan **Discord Node** dengan tipe koneksi *Webhook* untuk mengirim pesan tanpa perlu konfigurasi bot yang rumit.
> - **Dynamic Expressions (`{{ ... }}`):** menyusun isi pesan secara otomatis dengan menggabungkan teks biasa dan data variabel dari node-node sebelumnya.
> - **Cross-Node Referral:** mengambil parameter spesifik dari node yang berbeda di dalam *canvas* (seperti mengambil *Assessment ID* dari *header* node `GetDataFromWarehouse`).

⚙️ Konfigurasi Discord Node

Parameter	Nilai
Node Type	`Discord`
Connection Type	`Webhook`
Credential Status	`Create New Credential`
Credential Name	`n8n Quickstart S1 Discord Webhook account`
Webhook URL	`https://learn.app.n8n.cloud/webhook/courses/n8n-quickstart/notify-team`
Operation	`Send a Message`

---

8. Scheduling the Workflow — Part 8

`#n8n` `#workflow` `#automation` `#scheduling` `#schedule-trigger` `#learning-notes`

> 💡 **Konsep Utama**
>
> - **Otomatisasi Penjadwalan (`Schedule Trigger`):** memungkinkan *workflow* berjalan otomatis di latar belakang pada tanggal, waktu, atau interval tertentu tanpa pemicu manual.
> - **Dukungan Multi-Trigger:** satu *workflow* dapat memiliki lebih dari satu *trigger*. *Manual Trigger* tetap dipertahankan untuk pengujian mandiri (*on-demand testing*), sedangkan *Schedule Trigger* digunakan untuk jadwal otomatis rutin.
> - **Pentingnya Zona Waktu (*Timezone Sensitivity*):** akurasi waktu eksekusi sangat bergantung pada pengaturan zona waktu di *Workflow Settings* atau instansi n8n.

⚙️ Konfigurasi Node `TriggerMondays9am`

- **Node Type:** `Schedule Trigger`
- **Node Name:** `TriggerMondays9am`
- **Connected To:** node HTTP Request `GetOrdersFromWarehouse`

Parameter	Nilai Konfigurasi	Keterangan
Trigger Interval	`Weeks`	Interval eksekusi berbasis mingguan.
Weeks Between Triggers	`1`	Dijalankan setiap 1 minggu sekali.
Trigger on weekdays	`Monday`	Dijalankan setiap hari Senin (hapus *Sunday* bawaan).
Trigger at Hour	`9am`	Pukul 09.00 pagi.
Trigger at Minute	`0`	Menit ke-0.

🕘 Zona Waktu & Syarat Eksekusi Otomatis

1. **Aturan Hierarki Timezone:** - *Schedule Trigger* pertama-tama akan memeriksa dan menggunakan zona waktu di **Workflow Settings**. - Jika zona waktu *workflow* tidak diatur, sistem akan *fallback* menggunakan zona waktu dari **n8n Instance**.
2. **Syarat Status Workflow (*Publish*):** - *Schedule Trigger* **TIDAK AKAN** berjalan otomatis jika *workflow* masih berupa draf (*unpublished*). - *Workflow* harus **dipublikasikan / diaktifkan (*Publish*)** terlebih dahulu agar pemicu jadwal bekerja di latar belakang.

> 🚨 **Catatan Penting**
>
> - **Best Practice Penamaan:** memberi nama node secara spesifik (seperti `TriggerMondays9am`) mempermudah keterbacaan alur kerja saat *workflow* makin kompleks.
> - **Langkah Selanjutnya:** setelah *workflow* siap dan terintegrasi penuh, tahap berikutnya adalah **Publish Workflow** dan mempelajari cara membaca **Execution Log**.

---

9. Publishing & Examining the Workflow — Part 9

`#n8n` `#workflow` `#automation` `#executions` `#settings` `#debugging` `#learning-notes`

> 💡 **Konsep Utama**
>
> - **Workflow Executions:** mewakili satu kali putaran jalannya *workflow* dari node pertama hingga terakhir.
> - **Execution Log & Debugging:** catatan historis untuk memantau status keberhasilan/kegagalan, memeriksa data aktual di setiap node, dan melakukan *troubleshooting* jika terjadi kendala.
> - **Workflow Settings Customization:** pengaturan khusus per-*workflow* yang dapat menimpa (*overwrite*) konfigurasi default global instansi n8n (seperti *timezone*, penyimpanan log, hingga *error handling*).
> - **Data Wrangling Elimination:** mengubah pekerjaan manual yang rumit (*data wrangling*) menjadi otomatisasi rutin 8-node yang berjalan konsisten di latar belakang.

📊 Detail Tabel pada Jendela Executions

Kolom Info	Deskripsi & Fungsi
Name	Nama *workflow* yang dieksekusi.
Started At	Stempel waktu (*timestamp*) saat eksekusi dimulai.
Status	Status eksekusi (`Waiting` , `Running` , `Succeeded` , `Cancelled` , `Failed`) & durasi waktu eksekusi.
Execution ID	ID pengenal unik untuk setiap sesi eksekusi.

> **Filtering Options**
>
> Riwayat eksekusi dapat disaring berdasarkan status (`Any Status` , `Success` , `Failed` , `Cancelled` , `Running` , `Waiting`) maupun spesifik *workflow*.

⚙️ Fitur Kunci pada Workflow Settings

Parameter Pengaturan	Fungsi & Konfigurasi Utama
Execution Logic	Logika eksekusi *multi-branch* (default yang direkomendasikan: `v1`).
Error Workflow	Mengarahkan ke *workflow* lain untuk mengirim notifikasi khusus jika *workflow* utama gagal.
Sub-workflow Call	Mengatur apakah *workflow* ini dapat dipanggil oleh node `Execute Sub-workflow` .
Timezone	Zona waktu spesifik *workflow* (sangat vital untuk akurasi pemicu `Schedule Trigger`).
Save Executions	Opsi menyimpan log eksekusi produksi (`Failed` / `Successful`) dan eksekusi `Manual` .
Save Execution Progress	Memungkinkan fitur *resume workflow* dari titik terhenti saat *error* (dapat memperlambat proses).
Redact Execution Data	Menyamarkan/menyembunyikan data sensitif dari log tampilan produksi/manual.
Estimated Time Saved	Estimasi waktu yang dihemat per eksekusi untuk menghitung ROI (*Return on Investment*) otomatisasi.

🗺️ Navigasi UI n8n

```
Editor Canvas ⟶ Tab Executions (Khusus Workflow Aktif)
Overview      ⟶ Tab Executions (Global All Workflows)
Editor UI (...) ⟶ Settings (Akses Workflow Settings)
```

---

10. Exporting & Importing Workflows — Part 10

n8n #workflow #automation #export #import #json #learning-notes`

> 💡 **Konsep Utama**
>
> - **Format Berkas JSON:** *workflow* n8n disimpan dan ditransfer dalam bentuk struktur data JSON lokal.
> - **Fleksibilitas Berbagi Data:** memudahkan proses pencadangan (*backup*), kolaborasi antar-pengembang, atau penggunaan templat *workflow* dari pihak luar.
> - **Dua Metode Utama:** ekspor/impor dapat dilakukan melalui **Menu Editor UI** (berkas/URL) maupun secara langsung di atas **Canvas** (salin/tempel pintasan *keyboard*).

⚙️ Metode Ekspor & Impor n8n

Jenis Akses	Metode Ekspor (Save/Share)	Metode Impor (Load/Add)
Via Menu Editor UI *(Titik Tiga Kanan Atas)*	Pilih **Download** → menyimpan *workflow* sebagai berkas `.json` di komputer.	Pilih **Import from File** *(berkas JSON)* atau **Import from URL** *(tautan publik)*.
Via Editor Canvas *(Pintasan Keyboard)*	Block seluruh node → tekan `Ctrl+C` (Windows) / `CMD+C` (Mac) untuk menyalin kode JSON.	Tempelkan langsung kode JSON di area kosong *canvas* menggunakan `Ctrl+V` / `CMD+V`.

> 🚨 **Catatan Penting**
>
> - **Kredensial Keamanan:** saat mengekspor *workflow* ke bentuk JSON, n8n secara otomatis **TIDAK** menyertakan data kredensial sensitif (seperti *API key* atau kata sandi). Pengguna yang mengimpor *workflow* tersebut harus mengonfigurasi ulang kredensial mereka sendiri.
> - **Canvas Shortcut:** metode `Ctrl+C` dan `Ctrl+V` di atas *canvas* adalah cara tercepat untuk memindahkan sebagian atau seluruh node antar-*tab workflow* yang berbeda.

PART 2 — Working with Data

1. Working with Data & Data Structures in n8n

`#n8n` `#javascript` `#data-structure` `#array` `#json`

💡 Konsep & Struktur Data Utama

- **Fungsi Utama n8n (ETL):** bekerja seperti alat *Extract, Transform, Load* di mana data mengalir antar node dalam format terstruktur yang bisa dibaca dan diinterpretasikan oleh setiap node.
- **Format Standar Data:** seluruh data yang dikirim antar node dalam n8n berbentuk **array of JSON objects** (kumpulan objek JSON), di mana setiap elemen di dalamnya disebut sebagai **items**.
- **Arrays (Larik):** daftar nilai berurutan yang dimulai dari indeks 0.
- Contoh: `["Leonardo", "Michelangelo", "Donatello", "Raphael"]` — `Donatello` berada di indeks `2`.
- **Objects (Objek):** struktur data yang menyimpan pasangan kunci-nilai (*key-value pairs*).

Contoh Object

```
{
  "name": "Alice",
  "email": {
    "personal": "alice@home.com",
    "work": "alice@company.com"
  }
}
```

---

2. Creating and Referencing Data with the Code Node

`#n8n` `#javascript` `#data-structure` `#array` `#json` `#code-node` `#data-referencing`

💡 Pembuatan Data Manual di Code Node

- **Format Standar & Auto-Wrap:** setiap objek di dalam array n8n dibungkus dengan kunci `json`. Pada versi n8n modern, `json` ini ditambahkan secara otomatis jika kamu mengembalikan (*return*) array of objects biasa.
- **Nested Objects (Objek Bersarang):** digunakan untuk mengelompokkan data yang lebih detail atau sub-properti. Dibuat dengan membungkus pasangan kunci-nilai ke dalam kurung kurawal `{}` tambahan (contoh: pemisahan email personal dan work).
- **Contoh Pembuatan Data:**
- Membuat variabel array kontak (Alice dan Bob) yang berisi properti `name` dan `email` (`personal` & `work`).

- Mengembalikan (*return*) variabel tersebut agar diproses ke node berikutnya.

Contoh kode pembuatan data

```
const contacts = [
  {
    name: 'Alice',
    email: {
      personal: 'alice@home.com',
      work: 'alice@company.com',
    },
  },
  {
    name: 'Bob',
    email: {
      personal: 'bob@home.com',
      work: 'bob@company.com',
    },
  },
];

return contacts;
```

n8n otomatis membungkus setiap objek menjadi `{ json: { ... } }`.

⚙️ Referensi Data Antar Node (Code Node & Upstream Node)

- `$input.all()`: mengambil seluruh data items yang masuk dari node tepat sebelumnya untuk dimanipulasi (misal: menambah kolom baru `items[0].json.workEmail = ...`).
- `$('NodeName')`: mengambil data dari node yang letaknya lebih jauh di atas (*upstream*).

```
// Menambah field baru dari node sebelumnya
const items = $input.all();
items[0].json.workEmail = items[0].json.email.work;

return items;
```

🔍 Opsi Pengambilan Item pada `$('NodeName')`

Opsi	Fungsi	Contoh
`.first()`	Mengambil item pertama saja. Dipakai jika output node hanya 1 item atau hanya butuh 1 hasil.	`$('NodeName').first().json.field`
`.last()`	Mengambil item paling terakhir.	`$('NodeName').last().json.field`

Opsi	Fungsi	Contoh
`.all()`	Mengambil seluruh item sebagai array.	`$('NodeName').all()`
`.item`	Mengambil item pada posisi/indeks yang sama dengan item yang sedang diproses. Dipakai saat 2 node memiliki jumlah item sama.	`$('NodeName').item.json.field`

📌 Best Practices Penamaan Node

- **Deskriptif & Self-Documenting:** ubah nama bawaan (seperti `HTTP Request 2`) menjadi nama yang menjelaskan fungsinya (seperti `GetCountries` atau `CalculateTotal`).
- **Gunakan CamelCase / PascalCase:** menghilangkan spasi pada nama node mempermudah dan memperbersih penulisan ekspresi kode saat dipanggil lewat `$('NamaNode')`.
- **Sticky Notes:** manfaatkan sticky notes di kanvas untuk menjelaskan alur workflow secara keseluruhan.

---

3. HTML & XML Data Processing in n8n

`#n8n` `#html-node` `#xml-node` `#web-scraping` `#data-transformation`

💡 Penggunaan HTML Node (Web Scraping)

- **Fungsi Utama:** mengekstrak konten atau informasi terstruktur dari halaman web (*web scraping*) menggunakan *CSS Selector*.
- **Konfigurasi Penting:**
- **Operation:** *Extract HTML Content*.
- **Source Data & Property:** mengambil sumber data HTML (misal dari node *HTTP Request*) ke dalam properti JSON yang sesuai.
- **Extraction Values:** menentukan nama kolom (*Key*) dan jalur elemen HTML target menggunakan *CSS Selector* (contoh: `.post .item-title a` untuk mengambil judul artikel).
- **Toggle Return Array:** diaktifkan jika ingin mengambil seluruh elemen/kemunculan di halaman, atau dimatikan jika hanya butuh hasil pertama saja.

⚙️ Penggunaan XML Node (Konversi Format)

- **Fungsi Utama:** melakukan konversi bolak-balik format data antara **XML ke JSON** dan **JSON ke XML**.
- **Kegunaan:** sangat berguna saat kamu perlu menjembatani atau mengirimkan data antar *web service* atau sistem lama (*legacy system*) yang mewajibkan format XML, sementara data di n8n berformat JSON.

---

4. Date, Time & Interval Data in n8n

`#n8n` `#date-time` `#luxon` `#wait-node` `#workflow-automation`

> 💡 **Tipe Data Tanggal & Waktu**

- **Format yang didukung:** `DATE`, `TIME`, `DATETIME`, `TIMESTAMP` (Unix / ms), dan `YEAR` dalam berbagai format.
- **Schedule Trigger:** menjadwalkan eksekusi workflow pada waktu, interval, atau durasi tertentu.
- **Wait Node:** menunda eksekusi workflow (berdasarkan waktu tertentu, interval, atau webhook call) untuk menghindari data tidak lengkap dari layanan yang lambat.

⚙️ Node & Metode Pengolahan Tanggal

1. **Date & Time Node (Pendekatan No-Code / Visual):** - **Fungsi:** mengubah format, menghitung/menambah tanggal, membulatkan (*round*), dan membandingkan tanggal. - **Aturan Pembulatan (Round):**
   - *Round Up:* memaksa tanggal naik ke awal bulan berikutnya tanpa memedulikan batas tengah bulan (misal: `10 April` langsung naik ke `1 Mei`).
   - *Round Down / Floor:* membuang sisa hari dan mundur ke awal bulan berjalan (misal: `28 April` jadi `1 April`).
   - *Round Nearest:* membulatkan ke bulan terdekat berdasarkan titik tengah bulan.
   - **Opsi Penting (`Include Input Fields`):** menjaga agar kolom data lama (`id`, `name`, `email`, dll.) tidak terhapus dan tetap ditampilkan bersama kolom hasil baru.
2. **Luxon Expressions (Pendekatan Kode / Fleksibel):** - Menggunakan pustaka JavaScript Luxon bawaan di n8n. - Diterapkan langsung dalam ekspresi (seperti di Edit Fields node) menggunakan metode `.toDateTime().plus()` atau `.toDateTime().minus()`.

📌 Ringkasan Latihan Workflow (Date Operations)

Node	Fungsi
GetCustomerData	Mengambil data pelanggan termasuk tanggal pembuatan (`created`).
RoundDate *(Date & Time Node)*	Membulatkan tanggal `created` ke atas (*Round Up*) ke awal bulan berikutnya dan menghasilkan kolom `newDate`.
CheckAfter1960 *(If Node)*	Membagi alur data berdasarkan kondisi apakah `newDate` setelah `1960-01-01`.
Add5days *(Date & Time Node — Cabang True)*	Menambahkan durasi 5 hari ke tanggal `newDate` menggunakan node visual.
Add20Years *(Edit Fields Node — Cabang False)*	Menambahkan 20 tahun ke tanggal `newDate` menggunakan ekspresi Luxon: `{{ $json.newDate.toDateTime().plus(20, 'years') }}`.

---

5. Binary Data in n8n

`#n8n` `#binary-data` `#http-request` `#edit-image` `#file-handling`

💡 Pengertian & Penanganan Binary Data

- **Apa itu Binary Data?** Berkas seperti gambar (*images*), PDF, audio, dan video yang direpresentasikan dalam sistem biner dan butuh konversi agar bisa dibaca/diproses.
- **Catatan Path File:**

> - Instalasi Cloud / Docker: gunakan `/tmp/filename` untuk path file.
> - Instalasi npm: gunakan `~/filename`.

⚙️ Core Nodes untuk Binary Data

Node	Fungsi
HTTP Request	Mentransfer berkas dari web *resources* dan API.
Read/Write Files from Disk	Membaca dan menulis file langsung (khusus n8n *self-hosted*).
Convert to File	Mengubah data input (misal JSON) menjadi file seperti CSV atau XLSX.
Extract From File	Mengonversi format biner ke JSON (misal mengambil teks dari PDF).
Edit Image	Melakukan manipulasi gambar seperti rotasi, *crop*, *resize*, dan *compositing*.

📌 Ringkasan Latihan (Downloading & Editing an Image)

- **GetDogURL (HTTP Request 1):** mengambil URL acak gambar anjing dari API (`https://learn.app.n8n.cloud/webhook/apis/dog-images`).
- **GetDogImage (HTTP Request 2):** mengunduh file biner gambar menggunakan ekspresi URL dari node sebelumnya (`{{ $json.message }}`).
- **Rotate180 (Edit Image Node):** memutar posisi gambar sebesar 180 derajat (terbalik secara vertikal/horizontal) menggunakan operasi *Rotate*.

---

6. Splitting Data in n8n

`#n8n` `#split-out` `#data-transformation` `#api-handling`

> 💡 **Mengapa Perlu Split Data?**
>
> - **Masalah:** saat mengambil data dari API, seringkali data datang dalam satu item tunggal yang di dalamnya berisi *nested array* (misalnya 1 objek dengan *field* `results` yang berisi 20 rekaman). Di n8n, ini terbaca sebagai 1 item saja.
> - **Solusi:** node hilir (*downstream nodes*) biasanya butuh memproses setiap rekaman secara terpisah, sehingga array tersebut harus dipecah.

⚙️ Cara Kerja Split Out Node

- **Fungsi Utama:** mengambil sebuah *field* yang berisi *array* dan mengekstrak setiap elemennya menjadi item yang berdiri sendiri. Ini adalah salah satu node transformasi data yang paling sering digunakan di n8n.
- **Pengaturan Utama:**
- **Field To Split Out:** diisi dengan nama *field* yang berisi *array* yang ingin diurai (contoh: `results`).
- Node akan membaca *field* tersebut, membuat satu item output per elemen *array*, dan secara opsional mempertahankan *field* lain dari item asli di samping item baru.

📌 Ringkasan Latihan (Split Pokemon)

- **Kondisi Awal:** data Pokémon dari API masuk sebagai satu item utuh berisi *array*.
- **SplitPokemon (Split Out Node):** diatur dengan `Field To Split Out` bernilai `results`.
- **Hasil Eksekusi:** berubah menjadi beberapa item terpisah (misal 5 item terpisah), di mana masing-masing item kini berdiri sendiri dan membawa *field* `name` serta `url` sendiri-sendiri untuk diproses oleh node berikutnya secara independen.

---

7. Merging Data in n8n

`#n8n` `#merge-node` `#data-transformation` `#sync`

> 💡 **Mengapa Perlu Merge Data?**
>
> - **Fungsi Utama:** menggabungkan data dari berbagai sumber berbeda menjadi satu kumpulan data tunggal (*single data set*).
> - **Kasus Penggunaan Umum:** menggabungkan data pelanggan dari CRM dengan data pesanan dari database, atau menyinkronkan data antar sistem (menghapus duplikat / memperbarui catatan).

🔄 Jenis Sinkronisasi Data

1. **One-way Sync:** data mengalir dari satu sumber utama ke satu atau beberapa tujuan (perubahan di sumber didorong ke tujuan, tapi tidak sebaliknya).
2. **Two-way Sync:** data mengalir secara dua arah antar sistem (perubahan di salah satu sistem disinkronkan ke sistem lainnya).

⚙️ Metode Penggabungan pada Merge Node (Join)

- **Append:** menggabungkan item secara sekuensial (semua item dari Input 1 diikuti oleh semua item dari Input 2).
- **Combine:** menggabungkan data dengan beberapa sub-opsi:
- *Merge by Fields:* mencocokkan item dari kedua input berdasarkan nilai *field* tertentu (mirip konsep *JOIN* pada database).
- *Merge by Position:* memasangkan item dari kedua input berdasarkan urutan posisinya (*index*).
- *Combine All:* membuat seluruh kemungkinan kombinasi item dari kedua input (*Cartesian product*).
- **Choose Branch:** memilih output dari salah satu input spesifik dan mengabaikan input lainnya.

> ⚠️ **Catatan Penting Saat Mengisi Parameter**
>
> Jika ingin mereferensikan nilai bersarang (*nested values*) pada parameter *Input 1 Field* dan *Input 2 Field* di *Merge node*, tuliskan *property key* menggunakan format **dot-notation** sebagai teks biasa (contoh: `name.common`), **bukan** sebagai ekspresi (jangan pakai `{{ $json.name.common }}`).

---

8. Looping & Batch Processing

`#n8n` `#looping` `#batch-processing` `#data-transformation` `#automation`

💡 Konsep Dasar Pengolahan Data di n8n

- Secara bawaan (*default*), node di n8n menerima item data (mirip dengan baris pada tabel), memprosesnya, dan mengeluarkan hasilnya.
- **Automatic Iteration:** sebagian besar node di n8n berjalan secara otomatis untuk setiap item.
- *Contoh:* jika node Slack terhubung ke node database yang mengembalikan 5 data pelanggan, node Slack akan otomatis berjalan 5 kali untuk mengirim 5 pesan tanpa perlu membuat perulangan (*loop*) secara manual.

⚙️ Eksekusi Sekali (Executing Nodes Once)

- Jika tidak ingin node memproses *semua* item yang diterima, aktifkan opsi **Execute Once** pada tab *Settings* di node tersebut.
- Node hanya akan memproses **item pertama** dan mengabaikan sisa item lainnya.

🔁 Membuat Loop Manual & Node Exceptions

1. Kapan Harus Membuat Loop Manual?

n8n melakukan iterasi otomatis untuk sebagian besar node, tetapi ada beberapa **pengecualian** di mana node tidak otomatis mengiterasi semua item:

- **Code Node** dalam mode *Run Once for All Items*.
- **HTTP Request:** saat menangani *pagination* dari API, loop manual diperlukan untuk mengambil data halaman demi halaman.
- **RSS Read:** hanya berjalan sekali untuk URL yang diminta.
- **Execute Workflow Node** dalam mode *Run Once for All Items*.
- **Beberapa Node Database** (CrateDB, MS SQL, MongoDB, QuestDB, TimescaleDB): menjalankan operasi *insert* dan *update* hanya sekali.

2. Cara Membuat Loop Manual (Metode Klasik)

- Hubungkan *output* dari suatu node ke *input* node sebelumnya.
- Gunakan node **If** untuk memeriksa kondisi kapan perulangan harus berhenti (*stopping condition*).

📦 Memecah Data Menjadi Batch (Splitting Data into Batches)

Digunakan untuk mengolah volume data yang besar secara bertahap atau menghindari pembatasan panggilan API (*API rate limits*).

Menggunakan Node: Loop Over Items

- **Fungsi:** memecah data masuk menjadi beberapa *batch* dengan ukuran yang dapat dikonfigurasi, lalu memproses setiap *batch* melalui node-node yang terhubung.
- **Pengolahan Per Item:** set **Batch Size = 1** jika ingin memproses item satu per satu.

- **Kelebihan:** loop berhenti secara otomatis setelah seluruh data habis dibagi dan diproses. **Tidak memerlukan node If** untuk *stopping condition*.

---

9. Checking & Debugging Failed Workflows

`#n8n` `#debugging` `#executions-log` `#troubleshooting` `#error-handling`

📜 Executions Log (Log Eksekusi)

- **Fungsi:** alat utama n8n untuk melihat riwayat jalannya workflow dan melacak kesalahan (*error*).
- **Akses:** melalui panel kiri Editor UI atau tab *Executions* saat workflow terbuka.
- **Informasi yang Ditampilkan:**
- Nama workflow
- Waktu mulai eksekusi
- Status (*Succeeded, Failed, Running, Waiting,* atau *Cancelled*)
- Durasi eksekusi
- Execution ID

⚙️ Menyimpan Eksekusi Berhasil (Saving Successful Executions)

- **Default:** n8n *hanya* menyimpan eksekusi yang **gagal** (*failed*).
- **Cara Mengaktifkan Log Berhasil:** buka *Workflow Settings* → set **Save Successful Production Executions** ke **Yes**.
- **Kegunaan:** berguna untuk memverifikasi workflow berjalan dengan benar atau memeriksa data output dari eksekusi masa lalu.

🔍 Memeriksa & Memperbaiki Eksekusi Gagal

1. Inspecting Failed Executions

- Memilih eksekusi gagal akan membuka mode *read-only*.
- Node yang gagal ditandai dengan **warna merah**.
- Klik pada node merah untuk melihat rincian pesan *error* dan data yang diproses saat kegagalan terjadi.
- Gunakan tombol *toggle* **Editor | Executions** di bagian atas untuk berpindah dengan cepat antara mode edit dan peninjauan log.

2. Debugging in Editor (Pinning Data)

- Klik tombol **Debug in editor** pada eksekusi yang gagal untuk menyalin data eksekusi tersebut ke workflow editor.
- **Pinned Data:** data yang menyebabkan *error* akan otomatis terpasang (*pinned*) di tiap node.
- **Keuntungan:** kamu bisa menjalankan ulang node individual atau seluruh workflow menggunakan data persis yang menyebabkan *error* tanpa perlu memicu (*trigger*) eksekusi dari awal.
- **Langkah Akhir:** perbaiki masalahnya, tes terhadap *pinned data*, lalu lepas peniti data (*unpin*) setelah masalah selesai.

10. Catching & Handling Errors

`#n8n` `#error-handling` `#retry-on-fail` `#stop-and-error` `#error-trigger` `#automation`

> 💡 **Tingkatan Error Handling di n8n**
>
> n8n menyediakan 3 tingkatan penanganan *error*, dari yang paling spesifik (per node) hingga yang paling luas (seluruh workflow): 1. **Node-level Settings:** pengaturan penanganan *error* pada masing-masing node di tab *Settings*. 2. **Stop and Error Node:** node khusus untuk sengaja menghentikan workflow jika data tidak valid. 3. **Error Workflows:** workflow terpisah yang berjalan otomatis saat ada *production workflow* yang gagal untuk mengirimkan notifikasi.

⚙️ 1. Node-Level Error Handling

A. Pengaturan "On Error"

Mengontrol perilaku workflow saat node tertentu mengalami kegagalan:

Option	What Happens	When to Use It
Stop Workflow *(Default)*	Seluruh workflow langsung berhenti dan status eksekusi ditandai *failed*.	Saat node tersebut sangat krusial dan node setelahnya tidak boleh berjalan jika gagal.
Continue	Workflow tetap lanjut ke node berikutnya. Node yang gagal akan mengeluarkan data kosong (*empty data*).	Saat node bersifat opsional dan node setelahnya masih bisa bekerja tanpa data tersebut.
Continue Using Error Output	Node menambah 1 *output connector* baru berwarna merah (*error output*). Data sukses lewat output biasa, data gagal lewat output merah.	Saat ingin menangani *error* langsung di dalam workflow (misal: kirim notifikasi atau lewat jalur alternatif).

B. Retry on Fail

- **Fungsi:** otomatis mengulang eksekusi node jika terjadi kegagalan *transient* (*timeout*, *rate limit* API).
- **Konfigurasi:** terletak di tab *Settings* node. Bisa diatur jumlah pengulangan (hingga 5 kali) dan jeda waktu tunggu dalam milidetik (ms).

🛑 2. Stop and Error Node

- **Fungsi:** sengaja menghentikan workflow dan melempar *error* ketika kondisi tertentu terpenuhi.
- **Tipe Error:**
- **Error Message:** pesan teks kustom yang menjelaskan kesalahan.
- **Error Object:** objek struktur data untuk informasi kesalahan yang lebih detail.
- **Kasus Penggunaan:** dipasang setelah pengecekan validasi (misal node **If**) untuk menangkap data yang rusak/tidak sesuai dari API pihak ketiga sebelum masuk ke proses berikutnya.

🚨 3. Error Trigger & Error Workflows

Workflow terpisah di latar belakang yang menangani notifikasi kegagalan *production workflow*.

Langkah Pembuatan & Konfigurasi:

1. Buat workflow baru dan tambahkan node **Error Trigger** sebagai node pertama.
2. Sambungkan ke node notifikasi (misal: Discord, Slack, Gmail) untuk memberi tahu tim.
3. Simpan **Error Workflow** tersebut.
4. Buka workflow utama yang ingin dipantau → masuk ke **Workflow Settings** → pilih **Error Workflow** yang telah dibuat pada opsi *Error Workflow*.

> **Catatan Penting**
>
> - Workflow dengan node *Error Trigger* secara bawaan menggunakan dirinya sendiri sebagai penanganan kesalahan.
> - **Tidak bisa diuji secara manual:** hanya aktif saat eksekusi *production* (non-manual).
> - **Reusability:** satu *Error Workflow* dapat digunakan bersama oleh banyak workflow utama.

---

11. Hands-on Exercise: Workflow 1 — Merging Customer Data

`#n8n` `#hands-on` `#data-enrichment` `#merge-node` `#data-tables`

> **🎯 Tujuan Utama**
>
> Mengisi (*enrich*) kolom `region` dan `subregion` yang masih kosong pada Data Table `customers` secara otomatis dengan menggabungkan data internal (database) dan data eksternal (API n8n Academy).

🛠️ Garis Besar Alur Workflow

1. **Get Customers Data (Database Lokal)** - Mengambil 10 data pelanggan dari Data Table `customers`. - Kolom `region` dan `subregion` pada tahap ini masih bernilai kosong.
2. **Get Countries Data (API Eksternal)** - Mengambil daftar nama negara beserta informasi `region` dan `subregion` dari API n8n Academy (`https://learn.app.n8n.cloud/webhook/countries`).
3. **Merge Data (Pencocokan Data)** - Menggabungkan kedua data menggunakan node **Merge** (Mode: `Combine` → `Matching Fields`). - Mencocokkan kolom `customerCountry` (dari data pelanggan) dengan kolom `name` (dari API negara). - **Hasil:** data pelanggan kini diperkaya dengan informasi wilayah yang sesuai.
4. **Update Customers Table (Penyimpanan Kembali)** - Memperbarui (*update*) Data Table `customers` dengan data wilayah yang sudah lengkap. - Menggunakan acuan penanda **Conditions** (`id` Equals `{{ $json.id }}`) agar n8n memperbarui baris data yang tepat sesuai ID pelanggan masing-masing.

> **💡 Kunci Pembelajaran (Key Takeaways)**
>
> - **Data Enrichment:** teknik melengkapi data lokal yang belum lengkap menggunakan referensi dari API eksternal.
> - **Merge by Fields / Matching Fields:** cara mencocokkan 2 sumber data berbeda menggunakan nilai acuan (*key*) yang identik (contoh: Nama Negara).

- **Update Conditions:** syarat wajib saat memperbarui data di Data Table agar n8n memperbarui baris yang spesifik (*match by ID*), bukan menimpa data orang lain.

---

12. Hands-on Exercise: Workflow 2 (Part 1) — Retrieving Order Data

`#n8n` `#hands-on` `#http-request` `#data-table` `#merge-node` `#data-enrichment`

> 🎯 **Tujuan Utama**
>
> Mengambil data 30 transaksi (*order data*) dari *company warehouse* via API, lalu mengembangkannya (*enrichment*) dengan menggabungkan data profil pelanggan dari Data Table `customers` (termasuk `region` dan `subregion` yang sudah terisi di Workflow 1).

🛠️ Garis Besar Alur Workflow ("Generating Reports")

1. **GetOrderData (HTTP Request Node)** - Mengambil data pesanan dari API warehouse (`https://learn.app.n8n.cloud/webhook/courses/n8n-quickstart/company-data`). - Menerapkan otentikasi via **Header Auth** (`x-assessment-id`) untuk mengamankan panggilan API. - **Output:** 30 data pesanan (berisi `orderID` , `customerID` , `orderPrice` , `quantity` , `productCategory` , `orderDate` , dll).
2. **GetCustomers (Data Table Node — Cabang Paralel)** - Berjalan secara paralel dari *Manual Trigger*. - Mengambil seluruh baris data dari Data Table `customers` (resource: *Row*, operation: *Get*). - **Output:** 10 data pelanggan lengkap dengan kolom `region` dan `subregion` .
3. **Merge (Merge Node)** - Menghubungkan `GetOrderData` (Input 1) dan `GetCustomers` (Input 2). - **Pengaturan:** Mode `Combine` → `Matching Fields` (atau *Merge by Fields*). - **Key Pencocokan:** memadukan `customerID` (Input 1) dengan `customerID` (Input 2). - **Output Akhir:** 30 data transaksi yang kini sudah lengkap dengan informasi lokasi pelanggan (`country` , `region` , `subregion` , `email`).

> 💡 **Kunci Pembelajaran (Key Takeaways)**
>
> - **Kredensial Header Auth:** mengirimkan kunci identifikasi unik (`x-assessment-id`) di dalam *Header* HTTP Request agar server mengizinkan akses data.
> - **Cabang Paralel (Parallel Branching):** menjalankan node HTTP Request dan Data Table secara bersamaan dari satu *Trigger* sebelum akhirnya disatukan di node *Merge*.
> - **Relasi Data (One-to-Many Join):** satu pelanggan (`customerID`) bisa memiliki banyak transaksi. Node *Merge* otomatis mencocokkan `customerID` di setiap transaksi dengan profil pelanggan yang sesuai.

PART 3 — Building an AI Agent

> **Konteks**
>
> Bagian ini melanjutkan workflow **Customer service agent** yang sudah dibangun sebelumnya. Fokusnya: menambahkan **memory** agar agent mengingat percakapan, lalu memberi **tools** agar agent bisa membaca data nyata.

1. Masalah: Agent Tanpa Memory

Agent bisa mengobrol, tetapi coba lakukan ini: beri tahu namamu, tunggu respons, lalu tanya "*Siapa namaku?*" Agent tidak akan bisa menjawab. Ia mungkin meminta maaf atau menebak, tetapi tidak tahu apa yang kamu katakan 10 detik lalu.

Penyebabnya: **AI Agent tidak memiliki memory**. Setiap pesan diperlakukan sebagai percakapan yang benar-benar baru. Model hanya menerima pesan terbaru dan *system prompt*, tanpa riwayat sebelumnya.

Untuk *customer service agent*, ini masalah besar. Pelanggan bisa berkata "*Saya menelepon soal pesanan 5*" lalu melanjutkan dengan "*Bagaimana statusnya?*" Tanpa memory, agent sudah lupa pesanan mana yang dimaksud.

---

2. Uji Sendiri

Buka workflow **Customer service agent** yang sudah dibuat di unit sebelumnya.

1. Buka jendela chat dengan mengklik tombol **Chat** di bagian bawah *canvas*.
2. Ketik `Hi, my name is Nathan` lalu tekan **Enter**. Tunggu responsnya.
3. Sekarang ketik `What's my name?` lalu tekan **Enter**.

Agent tidak akan tahu namamu. Ia mungkin menjawab "*I don't have access to that information*" atau menebak. Apa pun jawabannya, jelas ia tidak mengingat apa yang kamu katakan satu pesan sebelumnya.

---

3. Menambahkan Simple Memory

n8n menyelesaikan masalah ini dengan **memory node**. Opsi paling sederhana adalah **Simple Memory**, yang menyimpan *window* pesan terbaru sehingga agent bisa melihat apa yang sudah dibicarakan dalam percakapan saat ini.

1. Di *canvas*, klik tombol **+** di bawah konektor **Memory** pada node AI Agent. Daftar opsi memory akan muncul.
2. Pilih **Simple Memory**.
3. Pengaturan default sudah cukup. Simple Memory menyimpan **5 interaksi terakhir** secara default — cukup untuk sebagian besar percakapan. Kamu bisa menambahkannya nanti jika agent perlu mengingat konteks lebih banyak.
4. Tutup node untuk kembali ke *canvas*.

---

4. Cara Kerja Memory

Simple Memory tidak memberi model sebuah "otak". Yang sebenarnya dilakukan adalah **menambahkan pesan-pesan terbaru ke prompt** setiap kali agent berjalan. Saat kamu mengirim "*What's my name?*", model menerima sesuatu seperti:

- **System prompt:** "*You are a customer service agent...*"
- **User message 1:** "*Hi, my name is Nathan*"
- **Assistant response 1:** "*Hello Nathan, how can I help you today?*"
- **User message 2:** "*What's my name?*"

Model bisa "mengingat" namamu karena ia melihat percakapan sebelumnya langsung di dalam prompt. Ukuran *window* (default 5 interaksi) mengontrol seberapa jauh konteks ini berlaku. Pesan yang lebih lama dibuang agar prompt tidak tumbuh terlalu besar.

> **Batasan Memory**
>
> - **Bukan penyimpanan permanen.** Jika percakapan berlangsung cukup lama, pesan awal akan keluar dari *window*.
> - **Hilang saat restart.** Simple Memory hanya tersimpan di memori, sehingga hilang ketika workflow berhenti atau instance n8n di-restart.
> - **Untuk produksi:** gunakan memory berbasis database seperti **Postgres Chat Memory** atau **Redis Chat Memory** agar percakapan tetap tersimpan setelah restart.

---

5. Menyimpan Workflow

Simpan workflow sebelum melanjutkan. Di unit berikutnya, agent akan diberi **tools** agar bisa benar-benar mencari data pelanggan dan pesanan, bukan menebak.

---

6. Memberi Agent Tools

Mengapa Tools Penting?

Saat ini agent bisa mengobrol dan mengingat apa yang kamu katakan, tetapi jika kamu bertanya "*What region is customer 10 in?*", ia akan mengarang jawaban atau mengaku tidak tahu. Model tidak memiliki akses ke datamu — ia hanya menghasilkan teks berdasarkan pola yang dipelajari saat training.

Tools mengubah itu. Tool adalah koneksi antara AI Agent dan sumber data eksternal. Ketika agent menerima pertanyaan yang tidak bisa dijawab dari pengetahuannya sendiri, ia bisa memutuskan untuk memanggil tool, membaca hasilnya, dan memakai informasi itu dalam respons. Kamu tidak memberi tahu kapan harus memakai tool: **agent memutuskan sendiri** berdasarkan pertanyaan dan deskripsi tool yang kamu berikan.

Di unit ini, kamu akan memberi customer service agent dua tools:

1. Membaca data pelanggan dari Data Table yang dibuat di Section 2.
2. Mengambil data pesanan dari endpoint API.

7. Menambahkan Tool Data Table `customers`

Tool pertama memberi agent akses ke data pelanggan: nama, negara, email, region, dan subregion.

1. Buka workflow **Customer service agent**.
2. Klik tombol **+** di bawah konektor **Tool** pada node AI Agent. Daftar tipe tool akan muncul.
3. Cari `Data Table` dan pilih **Data Table Tool**.
4. Di pengaturan tool, set **Operation** ke **Get**.
5. Untuk **Data Table**, pilih tabel **customers** yang dibuat di Section 2.
6. Ganti nama node menjadi **GetCustomers**.
7. Tutup node untuk kembali ke *canvas*.

> **Pentingnya Deskripsi Tool**
>
> Deskripsi tool adalah cara agent mengetahui kapan harus memakai tool tersebut. Deskripsi yang kabur membuat agent memanggil tool yang salah atau tidak memanggil sama sekali. Jelaskan secara spesifik **data apa yang dikembalikan** oleh tool tersebut.

---

8. Memperbarui System Prompt

Setelah agent punya tools, perbarui *system prompt* agar sesuai dengan kemampuannya.

1. Buka node **AI Agent** dan temukan kolom **System message** yang sudah diatur sebelumnya.
2. Ganti prompt dengan sesuatu seperti:

```plaintext
You are a customer service agent. You have access to a customer database and an order database.

When asked about customers, use the GetCustomers tool to look up their information. When asked about orders, prices, employees, or product categories, use the GetOrderData tool.

Be concise and helpful. If the data does not contain what the user is asking for, say so. Do not make up information.
```

3. Tutup node.

> **Mengapa perlu menyebut tool di prompt?**
>
> Memberi tahu agent tool mana yang dipakai untuk jenis pertanyaan tertentu sebenarnya tidak wajib (agent bisa menyimpulkannya dari deskripsi tool), tetapi ini membuat agent **lebih andal**. Ibarat memberi instruksi jelas kepada karyawan baru, daripada berharap ia menebak sendiri.

---

9. Menguji Agent dengan Tools

Buka jendela chat dan coba ajukan pertanyaan yang membutuhkan tools.

Contoh pertanyaan:

- `What region is customer 10 in?`
- `How many orders does customer 3 have?`
- `Who is the employee assigned to order 5?`
- `What is the total order value for all electronics orders?`

Perhatikan **panel log** di sisi kanan jendela chat. Kamu akan melihat agent memutuskan tool mana yang dipanggil, melakukan request, lalu memakai data yang dikembalikan untuk menyusun jawaban. Inilah perbedaan antara **LLM** (yang hanya bisa menghasilkan teks) dan **agent** (yang bisa mengambil tindakan dan memakai data nyata).

Coba juga pertanyaan yang membutuhkan **kedua tools**, seperti `What orders belong to customers in Europe?` Agent perlu mengecek tabel pelanggan untuk pelanggan di Eropa, lalu mencocokkannya dengan data pesanan. Prosesnya bisa beberapa langkah, dan kamu bisa melihat alur penalarannya di log.

---

10. Ketika Ada Masalah

AI agent tidak selalu benar. Kamu mungkin melihat agent sesekali memanggil tool yang salah, salah menafsirkan hasil, atau memberi jawaban yang sedikit berbeda untuk pertanyaan yang sama (ingat: **agent tidak deterministik**). Ini normal.

Beberapa hal yang membantu:

- **Tulis deskripsi tool yang jelas.** Agent mengandalkan deskripsi ini untuk memilih tool yang tepat.
- **Bersikap spesifik di system prompt** tentang apa yang boleh dan tidak boleh dilakukan agent.
- **Gunakan panel log untuk debugging.** Jika agent memberi jawaban salah, log akan menunjukkan tool mana yang dipanggil dan data apa yang dikembalikan.

---

11. Menyimpan Workflow & Langkah Lanjut

Simpan workflow. Sekarang kamu punya customer service agent yang bisa mengingat percakapan, mencari data pelanggan, dan mengambil data pesanan. Ini contoh dasar, tetapi sudah mencakup blok-blok utama: **trigger, agent, model, memory, dan tools**.

Dari sini kamu bisa menambahkan lebih banyak tools, menyempurnakan prompt, atau menghubungkan agent ke antarmuka chat langsung.

> **Belajar Lebih Lanjut**
>
> - Advanced AI documentation
> - AI workflow templates

📝 Bank Soal & Kuis QA101

> **Cara Membaca**
>
> Setiap soal berisi pertanyaan, pilihan jawaban, dan kunci jawaban di callout hijau beserta penjelasan singkat.

Kuis Getting Started

Soal 1 — Aplikasi tanpa node integrasi

Pertanyaan: Apa yang bisa kamu lakukan jika tidak ada node n8n untuk aplikasi/layanan yang ingin digunakan dalam workflow?

1. Meminta tim n8n membuatkan node
2. Membuat node sendiri
3. Menggunakan HTTP Request node

> ✅ **Jawaban: **Menggunakan HTTP Request node****
>
> HTTP Request adalah solusi serbaguna untuk memanggil API apa pun yang belum memiliki node khusus.
> *(Catatan: soal ini sebelumnya mendapat 0.0 — pastikan menjawab dengan opsi HTTP Request.)*

Soal 2 — Trigger Node

Pertanyaan: Manakah yang termasuk **Trigger Node**?

1. `Schedule node`
2. `Code node`
3. `Airtable node`
4. `Edit Fields node`

> ✅ **Jawaban: ** `Schedule node` **
>
> Hanya Schedule Trigger yang memulai workflow secara otomatis. Code, Airtable, dan Edit Fields adalah node pemrosesan biasa.

Soal 3 — Cara Code Node mengeksekusi kode

Pertanyaan: Bagaimana Code node mengeksekusi kode?

1. Kode dieksekusi sekali untuk semua item
2. Kode hanya bisa dijalankan oleh Schedule Trigger
3. Kode berjalan terus-menerus di latar belakang sampai dihentikan manual
4. Kode dieksekusi sekali untuk setiap item

> ✅ **Jawaban: **Kode dieksekusi sekali untuk semua item****

> Mode default Code node adalah *Run Once for All Items*. Mode *Run Once for Each Item* juga tersedia, tetapi bukan perilaku default.

Soal 4 — IF vs Switch

Pertanyaan: Apa perbedaan antara IF node dan Switch node?

1. Tidak ada, keduanya sama
2. IF mendukung dua jalur kondisi, Switch mendukung jalur kondisi tak terbatas
3. IF mendukung jalur kondisi tak terbatas, Switch mendukung dua jalur kondisi

> ✅ **Jawaban: **IF mendukung dua jalur, Switch mendukung banyak jalur****
>
> IF = percabangan `True` / `False`. Switch = banyak cabang sesuai aturan yang dibuat.

Soal 5 — Struktur data yang benar

Pertanyaan: Manakah struktur data yang benar?

1. `{[json: {jsonKeyName: 'keyValue'}]}`
2. `[{json: {jsonKeyName: 'keyValue'}}]`
3. `json: [jsonKeyName: 'keyValue']`

> ✅ **Jawaban: ** `[{json: {jsonKeyName: 'keyValue'}}]` **
>
> n8n menggunakan **array of objects**, dan setiap objek dibungkus dalam kunci `json`.

Soal 6 — Publish workflow dengan Schedule Trigger

Pertanyaan: Apa yang terjadi saat mem-publish workflow yang memiliki Schedule Trigger?

1. Workflow berjalan sekali langsung lalu berhenti sampai dipicu manual lagi
2. Workflow dibagikan publik agar pengguna n8n lain bisa mengeditnya
3. Workflow akan berjalan otomatis pada waktu terjadwal tanpa tindakan manual
4. Schedule Trigger masih perlu diaktifkan terpisah dari publish

> ✅ **Jawaban: **Workflow berjalan otomatis sesuai jadwal****
>
> Setelah dipublish, Schedule Trigger aktif di latar belakang dan menjalankan workflow sesuai jadwal.

Soal 7 — Penyebab error pada node

Pertanyaan: Apa saja yang bisa menjadi penyebab error pada node?

1. Kredensial node hilang atau salah
2. Parameter node tidak dikonfigurasi dengan benar
3. Kamu tidak memiliki izin untuk menggunakan node tersebut

> ✅ **Jawaban: **Semua poin di atas****

> Error bisa berasal dari kredensial, konfigurasi parameter, maupun izin akses.

Soal 8 — Struktur data n8n

Pertanyaan: Apa struktur data n8n?

1. Sebuah string JSON
2. Sebuah array of objects
3. Sebuah object tunggal
4. Sebuah array of arrays

> ✅ **Jawaban: **Array of objects****
>
> Data antar node selalu berupa array, di mana setiap elemen adalah objek `{ json: ... }`.

Soal 9 — Aksi node per unit data

Pertanyaan: Sebuah node n8n melakukan aksinya pada setiap _____ data yang masuk.

1. `key`
2. `array`
3. `workflow`
4. `item`

> ✅ **Jawaban: ** `item` ****
>
> Node n8n umumnya memproses data per **item** secara otomatis.

Kuis Working with Data

Soal 10 — Referensi node yang tidak terhubung langsung

Pertanyaan: Kamu perlu mereferensikan field `region` dari node bernama `GetCountries` yang **tidak terhubung langsung** ke node saat ini. Ekspresi mana yang benar?

1. `{{ $json.region }}`
2. `{{ $('GetCountries').item.json.region }}`
3. `{{ $input('GetCountries').json.region }}`
4. `{{ $json.GetCountries.region }}`

> ✅ **Jawaban: ** `{{ $('GetCountries').item.json.region }}` ****
>
> Gunakan `$('NamaNode')` untuk mengambil data dari node lain, lalu `.item` untuk item pada indeks yang sama.

Soal 11 — Menambah 20 tahun pada tanggal

Pertanyaan: Di Edit Fields node, ekspresi mana yang menambahkan 20 tahun ke field tanggal `customerSince`?

1. `{{ $json.customerSince + years(20) }}`
2. `{{ $json.customerSince.toDateTime().plus(20, 'years') }}`
3. `{{ DateTime.fromISO($json.customerSince) + 20 }}`
4. `{{ $json.customerSince.addYears(20) }}`

> ✅ **Jawaban: ** `{{ $json.customerSince.toDateTime().plus(20, 'years') }}` ****
> n8n memakai Luxon. Konversi dulu dengan `.toDateTime()`, lalu gunakan `.plus(20, 'years')`.

Soal 12 — Continue Using Error Output

Pertanyaan: Apa yang terjadi saat pengaturan *On Error* sebuah node diubah menjadi **Continue Using Error Output**?

1. Item gagal dilewati diam-diam dan hanya item sukses yang lanjut
2. Workflow berhenti dan error ditulis ke log
3. Output kedua muncul pada node dan item gagal diarahkan ke sana
4. Node mencoba ulang operasi hingga tiga kali sebelum gagal

> ✅ **Jawaban: **Output kedua muncul dan item gagal diarahkan ke sana****
> Node mendapat *error output* tambahan (berwarna merah) khusus untuk item yang gagal.

Kuis Building an AI Agent

Soal 13 — Workflow standar vs AI agent

Pertanyaan: Apa perbedaan utama antara workflow n8n standar dan workflow AI agent?

1. AI agent tidak membutuhkan trigger node
2. Workflow standar bersifat deterministik, AI agent tidak
3. Workflow standar tidak bisa memakai HTTP Request node
4. Workflow AI agent berjalan lebih cepat

> ✅ **Jawaban: **Workflow standar deterministik, AI agent tidak****
> Workflow standar mengikuti alur tetap. AI agent mengambil keputusan (misalnya memilih tool) sehingga hasilnya bisa bervariasi.

Soal 14 — AI Agent memilih tool

Pertanyaan: Bagaimana AI Agent memutuskan tool mana yang akan dipanggil?

1. Kamu harus mengonfigurasi kondisi untuk memberi tahu tool mana yang dipakai
2. Ia membaca deskripsi tool dan mencocokkannya dengan pertanyaan pengguna
3. Tool dipanggil sesuai urutan penambahannya ke canvas
4. Ia memanggil semua tool pada setiap pesan lalu memilih hasil terbaik

> ✅ **Jawaban: **Ia membaca deskripsi tool dan mencocokkannya dengan pertanyaan****
>
> Karena itu, deskripsi tool harus jelas dan spesifik.

Soal 15 — Fungsi system message

Pertanyaan: Apa yang dikontrol oleh *system message* (system prompt) pada AI Agent?

1. Ukuran window memory
2. Jumlah tool yang bisa dipanggil per pesan
3. Perilaku, kepribadian, dan instruksi agent
4. Model bahasa yang digunakan agent

> ✅ **Jawaban: **Perilaku, kepribadian, dan instruksi agent****
>
> System prompt menentukan peran, gaya jawaban, dan batasan agent.

Soal 16 — Kekurangan Simple Memory untuk produksi

Pertanyaan: Mengapa Simple Memory tidak direkomendasikan untuk penggunaan produksi?

1. Tidak berfungsi dengan model OpenAI
2. Hanya bisa mengingat satu pesan dalam satu waktu
3. Lebih mahal daripada memory berbasis database
4. Hanya menyimpan pesan di memori, sehingga hilang saat restart

> ✅ **Jawaban: **Hanya tersimpan di memori dan hilang saat restart****
>
> Untuk produksi, gunakan **Postgres Chat Memory** atau **Redis Chat Memory** agar percakapan persisten.

✅ Penutup

🎉 Course Complete

Selamat! Kamu telah menyelesaikan n8n Quickstart. Kamu memulai dari *canvas* kosong dan membangun AI agent yang bisa mencari data nyata serta menjawab pertanyaan. Sepanjang perjalanan, kamu belajar menghubungkan trigger dan HTTP request, mentransformasi dan menggabungkan data antar node, menangani error sebelum menjadi masalah, serta menghubungkan model AI ke tools eksternal.

🚀 Langkah Selanjutnya

n8n Quickstart dirancang agar kamu cepat produktif. Jika ingin lebih dalam, program **n8n Foundations** mencakup tiga kursus:

Kursus	Fokus
Essentials: Your First Workflows (N8N101)	Canvas, core nodes, alur data, dan membangun integrasi pertama dari nol.
Integrations: APIs and Connected Workflows (N8N102)	HTTP request, webhook, autentikasi API, logika IF/Switch/Loop, sub-workflow, Data Tables, dan error handling. Semua latihan memakai API praktik dari Academy, jadi tidak perlu akun eksternal.
In Practice: AI, Testing and Best Practices (N8N103)	Workflow berbasis AI dengan AI Agent node, testing & debugging dengan execution log dan Try/Catch, serta organisasi workflow dengan naming convention, version control, dan kesiapan produksi.

Selesaikan ketiga kursus Foundations untuk mendapatkan **badge n8n Foundations**. Ujian sertifikasi resmi n8n juga sedang disiapkan untuk memvalidasi kemampuanmu di seluruh program.

💬 Feedback & Komunitas

Kursus ini dirancang untuk memberi pengalaman belajar terbaik. Jika ada saran perbaikan — konten, tempo, latihan, atau hal lain — sampaikan kepada tim n8n.

- Kirim feedback
- **Komunitas online:** diskusi, tanya jawab, dan berbagi workflow di forum resmi community.n8n.io
- **Meetup & event lokal:** anggota komunitas dan ambassador mengadakan meetup untuk bertemu pengguna n8n lain, mengikuti talks/workshop, dan networking. Event mendatang diumumkan lewat kanal komunitas dan Luma page.

📌 Lampiran — Cheat Sheet Node

Node	Fungsi Utama	Catatan Penting
Manual Trigger	Menjalankan workflow secara manual saat pengujian.	Tetap dipertahankan meski sudah ada trigger otomatis.
Schedule Trigger	Menjalankan workflow sesuai jadwal.	Butuh workflow **Published**; ikuti timezone Workflow Settings.
HTTP Request	Memanggil API apa pun.	Solusi jika tidak ada dedicated node; atur method, header, auth.
Data Table	Database internal mirip spreadsheet.	Workflow & tabel harus satu workspace; gunakan Upsert agar tidak duplikat.
If	Percabangan 2 jalur (`True` / `False`).	Gunakan untuk logika if-then-else.
Switch	Percabangan banyak jalur.	Untuk lebih dari 2 kondisi.
Edit Fields (Set)	Memilih/menambah/menimpa field.	Kurangi payload sebelum masuk database.
Code	Menjalankan JavaScript/Python kustom.	Pilih *Run Once for All Items* atau *Run Once for Each Item*.
Discord	Mengirim pesan ke Discord.	Bisa via Webhook tanpa bot.
Split Out	Memecah array menjadi item terpisah.	Isi *Field To Split Out* dengan nama array.
Merge	Menggabungkan data dari beberapa sumber.	Mode Append/Combine/Choose Branch; nested key pakai dot-notation.
Loop Over Items	Memproses data per batch.	`Batch Size = 1` untuk per item; berhenti otomatis.
HTML	Web scraping dengan CSS Selector.	Aktifkan *Return Array* untuk banyak elemen.
XML	Konversi XML ↔ JSON.	Berguna untuk sistem legacy.
Date & Time	Mengubah/menghitung/membulatkan tanggal.	Aktifkan *Include Input Fields* agar kolom lama tetap ada.
Edit Image	Rotate, crop, resize, composite.	Bekerja pada binary data.
Error Trigger	Memulai *error workflow*.	Hanya aktif pada eksekusi production, tidak bisa dites manual.
AI Agent	Menjalankan agent dengan model, memory, dan tools.	Butuh system prompt yang jelas.
Simple Memory	Menyimpan window percakapan.	Default 5 interaksi; tidak persisten, gunakan Postgres/Redis untuk produksi.
Data Table Tool	Memberi agent akses ke Data Table.	Set operation `Get` ; beri nama deskriptif seperti `GetCustomers` .